Python 编程 · 第 1–3 章 复习

基础与字符串

按老师复习 PPT 逐页精讲

注释 · 数据类型 · 变量与输入 · 函数与模块 · 运算符 · 数学函数 · 字符串

2026年10月29日

目录

第0部分　这份复习讲什么

第1部分　考试怎么考 & 哪些是重点

第2部分　注释与数据类型（PPT 第3页）

第3部分　变量与输入（PPT 第4页）

第4部分　函数与模块（PPT 第5–6页）

第5部分　运算符与优先级（PPT 第7–8页）

第6部分　数学函数（PPT 第8页）

第7部分　字符串是什么（PPT 第9页）

第8部分　索引与切片（PPT 第10页）

第9部分　不可变、转义字符、三重引号（PPT 第11页）

第10部分　拼接与重复（PPT 第12页）

第11部分　字符串工具与格式化（PPT 第13页）

第12部分　易错点汇总（考前必看）

第13部分　自测题

第14部分　自测答案与讲解

第15部分　PPT 最后一页的动手练习（参考答案）

第16部分　一页速查表

所有示例代码均在 Python 3.13 中实际运行，“运行结果”为真实输出。

第0部分　这份复习讲什么

> **一句话概括：** 老师的复习 PPT 分两块——**基础篇**（程序是怎么跑起来的）和**字符串篇**（文字怎么取、怎么变）。这份资料按 PPT 的顺序，把每一页都掰开讲一遍，并补上 PPT 没写出来、但做题经常用到的细节。

PPT 页码	内容	本资料位置
第 3 页	注释、数据类型	第2部分
第 4 页	变量、`input()` 输入、类型转换	第3部分
第 5–6 页	函数、模块、形参与实参、返回值	第4部分
第 7–8 页	运算符、优先级	第5部分
第 8 页	数学函数（含 PPT 上留空的 `round(3.1415, 2)`）	第6部分
第 9 页	字符串是什么	第7部分
第 10 页	索引、切片	第8部分
第 11 页	不可变、转义字符、三重引号	第9部分
第 12 页	拼接 `+`、重复 `*`	第10部分
第 13 页	`upper/lower/len/str/min/max`、格式化	第11部分
第 14–15 页	总表、动手练习	第12–15部分

> **和第4章的关系：** 你昨天学的列表切片 `[start:stop:step]`，规则和这里的**字符串切片完全一样**。把这一章的索引切片吃透，第4章就轻松一半。

第1部分　考试怎么考 & 哪些是重点

> 说明：下面是 Python 入门课对这部分**最常见的考法**，具体以李老师的要求为准。老师如果划了重点，发给我，我再帮你对一遍。

常见题型：

1. **读代码写输出**：`7 / 2` 和 `7 // 2` 有什么不同？`s[1:4]` 是什么？`"Ha" * 3` 是什么？
2. **找错题**：`"3" + 3` 为什么报错？`input()` 的结果直接加 1 为什么报错？`s[0] = "p"` 为什么报错？
3. **概念题**：形参和实参的区别、没有 `return` 的函数返回什么、字符串为什么不可变。
4. **编程题**：输入名字打招呼、用函数算两数之和、把一句话切片并转大写（就是 PPT 最后一页那三道）。

重点划分：

等级	内容	要求
★★★ 必须掌握	`input()` 得到的是字符串，算数要先 `int()` / `float()`	闭着眼都能说出来
★★★ 必须掌握	字符串索引（含负索引）、切片“含头不含尾”	看到就能写出结果
★★★ 必须掌握	运算符 `+ - * / % // **` 和优先级	能算对
★★★ 必须掌握	`def` 定义函数、`return` 返回值	能写编程题
★★ 要会	数据类型 int/float/str/bool、`type()`	会判断
★★ 要会	字符串拼接、重复、`upper/lower/len/str`、`format` /f 字符串	会用
★★ 要会	转义 `\n` `\t`、三重引号、字符串不可变	会用、会解释
★ 理解即可	一元运算符、`import math`、`min/max` 比较字符	知道是什么

怎么用这份资料： 每段代码下面都有“运行结果”，全部在电脑上真实跑过；用到 `input()` 的，标签上写了模拟敲进去的内容。**先遮住结果自己猜，再对答案。**

第2部分　注释与数据类型（PPT 第3页）

2.1 注释：写给人看的便利贴

以 `#` 开头的内容叫**注释**，Python 运行时会直接跳过。它的作用是给自己和别人解释“这行代码在干嘛”。可以单独占一行，也可以写在代码后面。

代码

```
# 计算 3 个苹果的总价（这一行是注释，程序不执行）
price = 10   # 单价是 10 元
print(price * 3)
```

运行结果

```
30
```

> 小技巧：在 PyCharm 里选中几行按 `Ctrl + /`，可以一次性加上或去掉注释，调试时很好用。

2.2 四种基本数据类型

类型	名字	例子	一句话
`int`	整数	`3`、`18`、`-5`	没有小数点的数
`float`	浮点数（小数）	`3.5`、`9.9`、`1.0`	有小数点的数
`str`	字符串	`"苹果"`、`'hello'`、`"18"`	引号包起来的文字
`bool`	布尔值	`True`、`False`	只有真和假两种（首字母大写）

用 `type()` 可以查看任何东西是什么类型：

代码

```
print(type(18))
print(type(9.9))
print(type("苹果"))
print(type(True))
print(type("18"))
```

运行结果

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'str'>
```

> **重点中的重点：** `"18"` 带引号，所以它是**字符串**，不是数字！类型不同，同一个符号做的事也不同：

代码

```
print("3" + "3")
print(3 + 3)
print("3" * 3)
```

运行结果

```
33
6
333
```

- `"3" + "3"`：两个字符串，`+` 是**拼接** → `33`
- `3 + 3`：两个整数，`+` 是**加法** → `6`
- `"3" * 3`：字符串乘整数，`*` 是**重复** → `333`

字符串和数字**不能直接相加**，Python 会报错：

代码

```
print("3" + 3)
```

运行结果（报错）

```
TypeError: can only concatenate str (not "int") to str
```

第3部分　变量与输入（PPT 第4页）

3.1 变量：贴了标签的盒子

`name = "小明"` 的意思是：把 `"小明"` 放进一个盒子，盒子上贴标签 `name`。之后用 `name` 就能取出里面的东西。`=` 是“**放进去**”，不是数学里的“等于”。

代码

```
name = "小明"
age = 18
print(name, age)
age = 19          # 盒子里换了新东西
print(name, age)
```

运行结果

```
小明 18
小明 19
```

变量命名规则（考试会考）：

- 只能用**字母、数字、下划线**，而且**不能以数字开头**：`age1` ✓，`1age` ×
- 不能有空格：`my_name` ✓，`my name` ×
- 区分大小写：`Age` 和 `age` 是两个不同的变量
- 不能用 Python 的关键字（如 `if`、`for`、`def`、`True`）
- 最好也别用内置函数名当变量名（如 `str`、`len`、`list`），否则会把原来的功能“挡住”——你的作业里就有这个问题，见作业解析

3.2 input()：向用户提问

`input("提示语")` 会在屏幕上显示提示语，然后**停下来等用户打字**，用户按回车后，把打的内容交给你。

代码

```
name = input("你叫什么名字？")
print("你好，" + name)
```

运行结果　·　模拟键盘输入：小明

```
你叫什么名字？小明
你好，小明
```

> **全章最容易错的一点：`input()` 得到的永远是字符串**，哪怕用户输入的是数字！

代码

```
age = input("你几岁? ")
print(type(age))
print(age + "1")
```

运行结果 · 模拟键盘输入：18

```
你几岁? 18
<class 'str'>
181
```

用户输入 `18`，但 `age` 其实是字符串 `"18"`，所以 `age + "1"` 是拼接，得到 `181`，而不是 19。如果直接 `age + 1`，会报错：

代码

```
age = input("你几岁? ")
print(age + 1)
```

运行结果（报错） · 模拟键盘输入：18

```
你几岁? 18
TypeError: can only concatenate str (not "int") to str
```

3.3 要算数字，先转换

用 `int()` 把字符串转成整数，用 `float()` 转成小数。通常直接包在 `input()` 外面：

代码

```
age = int(input("你几岁? "))
print(age + 1)
height = float(input("身高多少米? "))
print(height * 100, "厘米")
```

运行结果 · 模拟键盘输入：18、1.75

```
你几岁? 18
19
身高多少米? 1.75
175.0 厘米
```

> `int(input(...))` 从里往外读：先 `input()` 拿到字符串，再 `int()` 转成整数。
>
> 注意：`int("1.75")` 会报错，因为 `"1.75"` 不是整数的写法；有小数时要用 `float()`。

第4部分　函数与模块（PPT 第5–6页）

4.1 函数像榨汁机

函数就是**一台机器**：你放进去原料（输入），它处理后吐出结果（输出）。函数分两种：

- **预定义函数（内置函数）**：Python 已经做好了，直接用，比如 `print()`、`len()`、`abs()`、`max()`。
- **用户定义函数**：自己用 `def` 写的，把经常要做的事包起来，以后一句话就能调用。

代码

```
print(len("Python"))
print(abs(-7))
print(max(3, 9, 5))
```

运行结果

```
6
7
9
```

4.2 自己定义一个函数

定义函数的格式：

1. `def 函数名(参数):` ——注意最后的**冒号**
2. 下面要做的事**缩进 4 个空格**（PyCharm 里按 Tab 会自动缩进）
3. 用 `return` 把结果交回去

代码

```
def make_juice(fruit):
    return fruit + "汁"

print(make_juice("橙子"))
print(make_juice("西瓜"))
```

运行结果

```
橙子汁
西瓜汁
```

4.3 形参和实参（PPT 第6页）

- **形参**：定义函数时，括号里写的**名字**（像菜单上的“杯型”，只是个空位）。
- **实参**：调用函数时，真正传进去的**值**（像你说的“大杯”）。

代码

```
def greet(name):          # name 是形参
    return "你好，" + name

result = greet("小明")    # "小明" 是实参
print(result)
```

运行结果

```
你好，小明
```

函数可以有多个参数，用逗号隔开，调用时按顺序一一对应：

代码

```
def add(a, b):
    return a + b

print(add(3, 5))
print(add(10, 20))
```

运行结果

```
8
30
```

4.4 返回值：不写 return 就交回 None

`return` 把结果“交回去”给调用的地方；没写 `return` 的函数，默认交回 `None`（表示“什么都没有”）。

代码

```
def say_hi():
    print("嗨！")

x = say_hi()
print(x)
```

运行结果

```
嗨！
None
```

> **易混：`print` 和 `return` 不一样。** `print` 是把东西**显示在屏幕上**；`return` 是把结果**交还给程序**，让你能存进变量、继续计算。上面 `say_hi()` 屏幕上显示了“嗨！”，但交回的是 `None`。

4.5 模块像工具箱

模块是别人整理好的一箱工具。用 `import 模块名` 把工具箱搬进来，再用 `模块名.工具名` 拿工具。最常用的是数学模块 `math`：

代码

```
import math
print(math.sqrt(16))
print(math.pi)
print(math.floor(3.7))
print(math.ceil(3.2))
```

运行结果

```
4.0
3.141592653589793
3
4
```

工具	作用
`math.sqrt(x)`	平方根（结果是小数）
`math.pi`	圆周率 π
`math.floor(x)`	向下取整（地板）
`math.ceil(x)`	向上取整（天花板）

第5部分　运算符与优先级（PPT 第7–8页）

5.1 算术运算符

PPT 上列了 `+ - * / %`，做题时还经常遇到 `//` 和 `**`，一起记：

符号	含义	例子	结果
`+`	加	`7 + 3`	`10`
`-`	减	`7 - 3`	`4`
`*`	乘	`7 * 3`	`21`
`/`	除（**结果永远是小数**）	`6 / 2`	`3.0`
`%`	取余数	`7 % 3`	`1`
`//`	整除（只要商的整数部分）	`7 // 3`	`2`
`**`	乘方	`2 ** 3`	`8`

代码

```
print(7 + 3)
print(7 - 3)
print(7 * 3)
print(7 / 3)
print(6 / 2)
print(7 % 3)
print(7 // 3)
print(2 ** 3)
```

运行结果

```
10
4
21
2.3333333333333335
3.0
1
2
8
```

> **用披萨记 `/`、`//`、`%`：** 7 块披萨分给 3 个人——`7 / 3` 是每人“理论上”分 2.33 块；`7 // 3` 是每人实际拿 **2** 块整的；`7 % 3` 是最后**剩下 1** 块。

`%` 最常见的用法是判断奇偶：一个数 `% 2` 等于 0 就是偶数，等于 1 就是奇数。

代码

```
print(10 % 2)
print(7 % 2)
```

运行结果

```
0
1
```

5.2 一元运算符

只对**一个数**动手的运算符：`-x` 表示取相反数，`+x` 表示正数（基本没变化）。就像温度“-5 度”。

代码

```
x = 5
print(-x)
print(+x)
print(-(-x))
```

运行结果

```
-5
5
5
```

5.3 赋值运算符

- `=`：把右边的值放进左边的变量。
- `+=`：在原来的基础上加。`x += 1` 等于 `x = x + 1`。
- 同理还有 `-=`、`*=`、`/=`。

代码

```
x = 5
x += 1
print(x)
x -= 2
print(x)
x *= 3
print(x)
```

运行结果

```
6
4
12
```

逐行跟踪：`x = 5` → `x += 1` 变成 6 → `x -= 2` 变成 4 → `x *= 3` 变成 12。

5.4 优先级：先算谁

> **口诀：括号 → 乘方 → 乘除（含 `//` `%`）→ 加减。** 同一级从左往右算。拿不准就**加括号**，最保险。

代码

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(10 - 4 / 2)
print(2 * 3 ** 2)
```

运行结果

```
14
20
8.0
18
```

- `2 + 3 * 4`：先乘，3×4=12，再加 2 → 14
- `(2 + 3) * 4`：括号先算 → 20
- `10 - 4 / 2`：先除，4/2=2.0，10-2.0 → `8.0`（有 `/` 就变成小数）
- `2 * 3 ** 2`：乘方优先，3^2=9，再乘 2 → 18

第6部分　数学函数（PPT 第8页）

这几个是**内置函数**，不用 `import` 就能直接用：

代码

```
print(abs(-5))
print(pow(2, 3))
print(round(3.6))
print(round(3.1415, 2))
print(sum([1, 2, 3]))
```

运行结果

```
5
8
4
3.14
6
```

函数	作用	例子
`abs(x)`	绝对值	`abs(-5)` → `5`
`pow(x, y)`	x 的 y 次方，和 `x ** y` 一样	`pow(2, 3)` → `8`
`round(x)`	四舍五入到整数	`round(3.6)` → `4`
`round(x, n)`	保留 n 位小数	`round(3.1415, 2)` → `3.14`
`sum(一组数)`	求和	`sum([1, 2, 3])` → `6`

PPT 第8页右下角留空的那道题： `round(3.1415, 2)` 的答案是 **`3.14`**。第二个参数 2 表示“保留 2 位小数”。

冷知识（偶尔会考）： Python 的 `round` 遇到“正好是 .5”时，会取**离它最近的偶数**，所以 `round(2.5)` 是 2，不是 3。平时做题遇到 `.5` 要特别小心。

代码

```
print(round(2.5))
print(round(3.5))
print(round(4.5))
```

运行结果

```
2
4
4
```

第7部分　字符串是什么（PPT 第9页）

用**单引号或双引号**包起来的文字就是字符串。单双引号效果一样，但前后要配对。

代码

```
name = "Python"
word = 'hello'
print(name, word)
print(type("3"))
```

运行结果

```
Python hello
<class 'str'>
```

字符串就像**一串珠子**：每个字符是一颗珠子，顺序不能乱。它是一个“对象”，自带很多好用的方法（第11部分讲）。

> 再强调一次：引号里的数字也是字符串，`"3"` 不能直接和 `3` 相加。

第8部分　索引与切片（PPT 第10页）

8.1 索引：取出一个字

和列表一样，**从 0 开始数**，也可以用负数**从后往前数**：

字符	P	y	t	h	o	n
正索引	0	1	2	3	4	5
负索引	-6	-5	-4	-3	-2	-1

代码

```
s = "Python"
print(s[0])
print(s[1])
print(s[-1])
print(s[-2])
print(len(s))
```

运行结果

```
P
y
n
o
6
```

`s[-1]` 就是最后一个字，不用先数长度，非常方便。

8.2 切片：取出一段

格式是 `s[开始:结束]`，**含头不含尾**——开始的位置取得到，结束的位置取不到。省略开始就是从头，省略结束就是到尾。

代码

```
s = "Python"
print(s[0:2])
print(s[2:])
print(s[:3])
print(s[1:4])
print(s[-3:])
print(s[::-1])
```

运行结果

```
Py
thon
Pyt
yth
hon
nohtyP
```

- `s[0:2]`：索引 0、1 → `Py`

- `s[2:]`：从索引 2 到最后 → `thon`
- `s[:3]`：前 3 个 → `Pyt`
- `s[1:4]`：索引 1、2、3 → `yth`
- `s[-3:]`：最后 3 个 → `hon`
- `s[::-1]`：倒过来 → `nohtyP`

> **火车比喻（PPT 原话）：** 火车车厢从 0 号开始编号，**索引是走进某一节车厢，切片是截下连续几节车厢。** `[a:b]` 截下来的车厢数正好是 **b − a** 节。

第9部分　不可变、转义字符、三重引号（PPT 第11页）

9.1 字符串不能改单个字

字符串是**不可变**的：想把 `"Python"` 的第一个字母改成小写，不能直接改：

代码

```
s = "Python"
s[0] = "p"
```

运行结果（报错）

```
TypeError: 'str' object does not support item assignment
```

只能**重新造一个新字符串**，再整个赋值给变量：

代码

```
s = "Python"
s = "p" + s[1:]
print(s)
```

运行结果

```
python
```

`"p" + s[1:]` 的意思是：新字母 `p` 拼上原来从索引 1 开始的部分 `ython`。**这个技巧作业第3章练习1要用到。**

> PPT 比喻：印好的书不能改一个字，只能重新印一本。

9.2 转义字符

有些字符没法直接打出来，就用反斜杠 `\` 加一个字母表示：

写法	含义
`\n`	换行
`\t`	制表符（像按一下 Tab，用来对齐）
`\'`	单引号本身
`\"`	双引号本身
`\\`	反斜杠本身

代码

```
print("第一行\n第二行")
print("姓名\t年龄")
print("小明\t18")
print('I\'m Tom')
print("他说：\"你好\"")
```

运行结果

```
第一行
第二行
姓名	年龄
小明	18
I'm Tom
他说："你好"
```

9.3 三重引号：直接写多行

用三个引号 `"""` 或 `'''` 包起来，可以在字符串里**直接换行**，不用写 `\n`：

代码

```
text = """第一行
第二行
第三行"""
print(text)
```

运行结果

```
第一行
第二行
第三行
```

第10部分　拼接与重复（PPT 第12页）

10.1 加号拼接

`+` 把两个字符串接成一个。**中间不会自动加空格**，要空格得自己加 `" "`：

代码

```
print("Hello" + "Python")
print("Hello" + " " + "Python")
age = 18
print("我今年" + str(age) + "岁")
```

运行结果

```
HelloPython
Hello Python
我今年18岁
```

> **易错：字符串不能直接拼数字。** 要先用 `str()` 把数字转成字符串（上面最后一行就是这么做的）。不转就报错：

代码

```
age = 18
print("我今年" + age + "岁")
```

运行结果（报错）

```
TypeError: can only concatenate str (not "int") to str
```

10.2 星号重复

`字符串 * 整数` 把字符串重复几遍。`"-" * 20` 常用来画分隔线：

代码

```
print("Ha" * 3)
print("-" * 20)
```

运行结果

```
HaHaHa
--------------------
```

第11部分　字符串工具与格式化（PPT 第13页）

11.1 常用工具

代码

```
print("abc".upper())
print("ABC".lower())
print(len("Python"))
print(str(123), type(str(123)))
print(min("abc"), max("abc"))
print(max("Hello"))
```

运行结果

```
ABC
abc
6
123 <class 'str'>
a c
o
```

写法	作用	例子 → 结果
`s.upper()`	全部转大写	`"abc".upper()` → `"ABC"`
`s.lower()`	全部转小写	`"ABC".lower()` → `"abc"`
`len(s)`	长度（几个字符，空格也算）	`len("Python")` → `6`
`str(x)`	把别的类型转成字符串	`str(123)` → `"123"`
`min(s)` / `max(s)`	最小 / 最大的字符	`max("abc")` → `"c"`

> 注意写法区别：`upper()` `lower()` 是字符串**自带的方法**，写成 `s.upper()`；`len()` `str()` `min()` `max()` 是**内置函数**，写成 `len(s)`。

min/max 怎么比较字符？ 按字符的编码大小比：数字 < 大写字母 < 小写字母，同类里按字母表顺序。所以 `max("Hello")` 是 `o`（小写 o 比大写 H 大，也比 e、l 大）。作业第3章练习2会用到。

11.2 字符串格式化：把变量填进句子

两种写法结果一样：

- `"{}今年{}岁".format(name, age)`：句子里的 `{}` 是空位，按顺序填进 `format` 括号里的值
- `f"{name}今年{age}岁"`：前面加 `f`，直接在 `{}` 里写变量名（更简洁，推荐）

代码

```
name = "小明"
age = 18
print("{}今年{}岁".format(name, age))
print(f"{name}今年{age}岁")
print("圆周率约等于{:.2f}".format(3.14159))
```

运行结果

```
小明今年18岁
小明今年18岁
圆周率约等于3.14
```

`{:.2f}` 的意思是“保留 2 位小数”；类似地，`{:.4}` 用在字符串上表示“**只保留前 4 个字符**”——作业第3章附加练习就靠这个。

第12部分　易错点汇总（考前必看）

编号	易错点	正确理解
1	`input()` 输入数字就是数字	**永远是字符串**，算数要 `int()` / `float()`
2	`"3" + 3` 能算出 6	报 TypeError；字符串和数字不能直接相加
3	`6 / 2` 结果是 `3`	`/` 结果永远是小数 `3.0`；要整数用 `//`
4	`7 // 2` 和 `7 % 2` 一样	`//` 是商（3），`%` 是余数（1）
5	`2 + 3 * 4` 是 20	先乘除后加减，是 14
6	函数没写 return 会报错	不报错，默认返回 `None`
7	`print` 就是 `return`	`print` 只显示，`return` 交回结果
8	第一个字是 `s[1]`	从 0 开始，第一个是 `s[0]`，最后一个是 `s[-1]`
9	`s[1:4]` 包含索引 4	含头不含尾，只有 1、2、3
10	可以写 `s[0] = "p"`	字符串不可变，只能整串重新赋值
11	`"Hello" + "World"` 中间有空格	没有，要自己加 `" "`
12	`round(2.5)` 是 3	是 2（取最近的偶数）
13	`True` 可以写成 `true`	首字母必须大写
14	变量名可以是 `1name`、`my name`	不能以数字开头，不能有空格

第13部分　自测题

> 先遮住答案自己写，写完再翻到第14部分对答案。

自测1（运算符） 写出每行输出：

代码

```
print(10 / 4)
print(10 // 4)
print(10 % 4)
print(2 ** 4)
```

自测2（索引与切片） 写出每行输出：

代码

```
s = "Banana"
print(s[2])
print(s[-1])
print(s[1:4])
print(s[::-1])
```

自测3（赋值运算符） 输出是什么？

代码

```
x = 3
x += 2
x *= 2
print(x)
```

自测4（input 陷阱） 运行时用户输入 `5`，输出是什么？

代码

```
a = input("输入一个数: ")
print(a * 2)
```

自测5（返回值） 输出是什么？

代码

```
def f(n):
    n = n + 1

print(f(5))
```

第14部分　自测答案与讲解

自测1：

运行结果

```
2.5
2
2
16
```

`10 / 4` 是 2.5（小数）；`10 // 4` 是商 2；`10 % 4` 是余数 2；`2 ** 4` 是 2 的 4 次方 16。

自测2：

运行结果

```
n
a
ana
ananaB
```

`"Banana"`：B0 a1 n2 a3 n4 a5。`s[2]` 是 n；`s[-1]` 是最后一个 a；`s[1:4]` 是索引 1~3 即 ana；`[::-1]` 倒过来。

自测3：

运行结果

```
10
```

x 从 3 开始，`+= 2` 变 5，`*= 2` 变 10。

自测4：

运行结果　·　模拟键盘输入：5

```
输入一个数：5
55
```

`a` 是字符串 `"5"`，`"5" * 2` 是重复两遍，得到 `55`，不是 10。

自测5：

运行结果

```
None
```

函数 `f` 里没有 `return`，所以返回 `None`。函数里改了 `n` 也没有交回来。

第15部分　PPT 最后一页的动手练习（参考答案）

PPT 第15页要求自己写三段小程序。建议你先自己在 PyCharm 里写，写不出来再看下面。

练习1：用 `input()` 问名字并打招呼

代码

```
name = input("请输入你的名字：")
print("你好，" + name + "！欢迎学习 Python")
```

运行结果　·　模拟键盘输入：小明

```
请输入你的名字：小明
你好，小明！欢迎学习 Python
```

练习2：用函数算两个数的和

代码

```
def add(a, b):
    return a + b

x = float(input("第一个数："))
y = float(input("第二个数："))
print("两数之和 =", add(x, y))
```

运行结果　·　模拟键盘输入：3、4.5

```
第一个数：3
第二个数：4.5
两数之和 = 7.5
```

用 `float()` 而不是 `int()`，这样输入小数也不会报错。

练习3：把一句话切片并转成大写

代码

```
sentence = "hello python world"
part = sentence[6:12]
print(part)
print(part.upper())
```

运行结果

```
python
PYTHON
```

`"hello python world"` 中，`python` 从索引 6 开始、到索引 11 结束，所以切片写 `[6:12]`（含头不含尾，结束位置要 +1）。

第16部分　一页速查表

知识点	一句话	例子
注释	`#` 开头，程序不执行	`# 计算总价`
数据类型	int 整数、float 小数、str 文字、bool 真假	`type("18")` → str
变量	贴标签的盒子，`=` 是放进去	`age = 18`
输入	`input()` 永远得到字符串	`int(input("年龄: "))`
函数	`def` 定义，形参占位、实参真值，`return` 交回	`def add(a, b): return a + b`
模块	`import` 搬工具箱	`import math; math.sqrt(16)`
算术	`+ - * / % // **`	`7 // 3` → 2，`7 % 3` → 1
赋值	`=`、`+=`、`-=`、`*=`	`x += 1`
优先级	括号 → 乘方 → 乘除 → 加减	`2 + 3 * 4` → 14
数学函数	`abs` `pow` `round` `sum`	`round(3.1415, 2)` → 3.14
索引	从 0 开始，-1 是最后一个	`"Python"[-1]` → n
切片	`[开始:结束]`，含头不含尾	`"Python"[0:2]` → Py
不可变	不能改单个字，只能整串重来	`s = "p" + s[1:]`
转义	`\n` 换行，`\t` 对齐	`"a\nb"`
拼接/重复	`+` 拼接，`*` 重复	`"Ha" * 3` → HaHaHa
工具	`upper` `lower` `len` `str` `min` `max`	`"abc".upper()` → ABC
格式化	`format()` 或 f 字符串	`f"{name}今年{age}岁"`